

Java Programing: An Introduction to OOP



Faculty of Computer Science and Mathematics, University of Kufa, Najaf, IRAQ.

1.7 Copy and Assign Java Arrays نسخ واسناد قيم المصفوفات في الجافا

هناك اكثر من طريقة لنسخ المصفوفات وهي:

1. Copying Arrays Using Assignment Operator

```
class CopyArray {
    public static void main(String[] args) {

        int [] numbers = {1, 2, 3, 4, 5, 6};
        int [] positiveNumbers = numbers;    // copying arrays

        for (int number: positiveNumbers) {
            System.out.print(number + ", ");
        }
    }
}
```

Output

1, 2, 3, 4, 5, 6

2. Using Looping Construct to Copy Arrays

Here, `for` loop is used to iterate through each element of `source` array. In each iteration, corresponding element of `source` array is copied to `destination` array.

```
import java.util.Arrays;

class ArraysCopy {
    public static void main(String[] args) {

        int [] source = {1, 2, 3, 4, 5, 6};
        int [] destination = new int[6];

        for (int i = 0; i < source.length; ++i) {
            destination[i] = source[i];
        }

        // converting array to string
        System.out.println(Arrays.toString(destination));
    }
}
```

Output

[1, 2, 3, 4, 5, 6]

3. Copying Arrays Using `arraycopy()` method

The `System` class contains `arraycopy()` method that allows you to copy data from one array to another. The `arraycopy()` method is efficient as well as flexible. The method allows you to copy a specified portion of the source array to the destination array.

Here,

- `src` - array you want to copy
- `srcPos` - starting position (index) in the `src` array
- `dest` - elements of `src` array will be copied to this array
- `destPos` - starting position (index) in the `dest` array
- `length` - number of elements to copy

```
// To use Arrays.toString() method
import java.util.Arrays;

class ArraysCopy {
    public static void main(String[] args) {
        int[] n1 = {2, 3, 12, 4, 12, -2};

        int[] n3 = new int[5];

        // Creating n2 array of having length of n1 array
        int[] n2 = new int[n1.length];

        // copying entire n1 array to n2
        System.arraycopy(n1, 0, n2, 0, n1.length);
        System.out.println("n2 = " + Arrays.toString(n2));

        // copying elements from index 2 on n1 array
        // copying element to index 1 of n3 array
        // 2 elements will be copied
        System.arraycopy(n1, 2, n3, 1, 2);
        System.out.println("n3 = " + Arrays.toString(n3));
    }
}
```

4. Copying Arrays Using copyOfRange() method

you can use `copyOfRange()` method defined in `java.util.Arrays` class to copy arrays. You do not need to create the destination array before this method is called. Visit this page to learn more about `copyOfRange()` method.

```
// To use toString() and copyOfRange() method
import java.util.Arrays;

class ArraysCopy {
    public static void main(String[] args) {

        int[] source = {2, 3, 12, 4, 12, -2};

        // copying entire source array to destination
        int[] destination1 = Arrays.copyOfRange(source, 0, source.length);
        System.out.println("destination1 = " + Arrays.toString(destination1));

        // copying from index 2 to 5 (5 is not included)
        int[] destination2 = Arrays.copyOfRange(source, 2, 5);
        System.out.println("destination2 = " + Arrays.toString(destination2));
    }
}
```

The output will be:

```
destination1 = [2, 3, 12, 4, 12, -2]
destination2 = [12, 4, 12]
```

5. Copying 2d Arrays Using Loop

Here's an example to copy irregular 2d arrays using loop:

```
import java.util.Arrays;

class ArraysCopy {
    public static void main(String[] args) {

        int[][] source = {
            {1, 2, 3, 4},
            {5, 6},
            {0, 2, 42, -4, 5}
        };

        int[][] destination = new int[source.length][];

        for (int i = 0; i < destination.length; ++i) {

            // allocating space for each row of destination array
            destination[i] = new int[source[i].length];

            for (int j = 0; j < destination[i].length; ++j) {
                destination[i][j] = source[i][j];
            }
        }

        // displaying destination array
        System.out.println(Arrays.deepToString(destination));
    }
}
```

the output will be:

```
[[1, 2, 3, 4], [5, 6], [0, 2, 42, -4, 5]]
```